

Android 第七章第二节教学讲义（系统数据交互实战版）

各位同学，今天咱们学习 Android 中与系统数据交互的核心技能 —— 比如操作手机通讯录、监听短信、调用相册、发送短信等。这些功能在实际 APP 中非常常见（比如社交 APP 添加联系人、电商 APP 验证短信），咱们用 "场景 + 代码" 的方式，保证小白也能跟着敲出效果！

一、操作联系人：添加、批量添加、查询

手机通讯录的数据是通过系统的 `ContentProvider` 暴露的，咱们可以通过 `ContentResolver` 来操作（和上节课学的跨应用数据交互原理一样）。

1. 权限准备（必须！）

操作联系人需要申请危险权限，在 `AndroidManifest.xml` 中先声明：

```
<!-- 声明联系人权限 -->
<uses-permission android:name="android.permission.READ_CONTACTS" /> <!-- 读联系人 -->
<uses-permission android:name="android.permission.WRITE_CONTACTS" /> <!-- 写联系人 -->
```

注意：Android 6.0 以上还需要在运行时动态申请（参考上节课的动态权限代码）。

2. 添加单个联系人

联系人数据比较特殊，需要操作 3 张系统表：

- `raw_contacts`：存联系人 ID（唯一标识）；
- `data`：存联系人详情（姓名、电话等，关联 `raw_contacts` 的 ID）；
- `mimetypes`：存数据类型（比如姓名是 `"vnd.android.cursor.item/name"`）。

```
// 添加联系人方法
private void addContact(String name, String phone) {
    // 获取ContentResolver
    ContentResolver resolver = getContentResolver();
```

```

// 1. 先在raw_contacts表中插入一条记录，获取新联系人的ID
Uri rawUri = Uri.parse("content://com.android.contacts/raw_contacts");
ContentValues rawValues = new ContentValues();
// 系统会自动生成ID，这里先传null
Uri newRawUri = resolver.insert(rawUri, rawValues);
// 解析新联系人的ID（从返回的URI中获取，如content://.../100）
long contactId = ContentUris.parseId(newRawUri);
// 2. 在data表中插入姓名
Uri dataUri = Uri.parse("content://com.android.contacts/data");
ContentValues nameValues = new ContentValues();
nameValues.put("raw_contact_id", contactId); // 关联联系人ID
nameValues.put("mimetype", "vnd.android.cursor.item/name"); // 数据类型：姓名
nameValues.put("data1", name); // 姓名内容
resolver.insert(dataUri, nameValues);
// 3. 在data表中插入电话
ContentValues phoneValues = new ContentValues();
phoneValues.put("raw_contact_id", contactId);
phoneValues.put("mimetype", "vnd.android.cursor.item/phone_v2"); // 数据类型：电话
phoneValues.put("data1", phone); // 电话号码
phoneValues.put("data2", "2"); // 电话类型（2=手机）
resolver.insert(dataUri, phoneValues);
Toast.makeText(this, "联系人添加成功", Toast.LENGTH_SHORT).show();
}

```

调用： addContact("张三", "13800138000");

3. 批处理添加联系人（高效方式）

添加多个联系人时，用ContentProviderOperation批量处理，效率更高：

```

private void batchAddContacts() {
    // 创建批处理操作列表
    ArrayList<ContentProviderOperation> operations = new ArrayList<>();
    // 添加第一个联系人：李四 13900139000
    addContactToBatch(operations, "李四", "13900139000");
    // 添加第二个联系人：王五 13700137000
    addContactToBatch(operations, "王五", "13700137000");
    try {
        // 执行批处理
        getContentResolver().applyBatch("com.android.contacts", operations);
    }
}

```

```

        Toast.makeText(this, "批量添加成功", Toast.LENGTH_SHORT).show();
    } catch (Exception e) {
        e.printStackTrace();
        Toast.makeText(this, "添加失败", Toast.LENGTH_SHORT).show();
    }
}
// 辅助方法：往批处理列表中添加一个联系人
private void addContactToBatch(ArrayList<ContentProviderOperation> ops, String name,
String phone) {
    // 1. 插入raw_contacts表
    ContentProviderOperation rawOp = ContentProviderOperation.newInsert(
        Uri.parse("content://com.android.contacts/raw_contacts"))
        .withValue(null, null) // 自动生成ID
        .build();
    ops.add(rawOp);
    // 2. 插入姓名（依赖上一步的ID，用@0表示第一个操作的结果）
    ContentProviderOperation nameOp = ContentProviderOperation.newInsert(
        Uri.parse("content://com.android.contacts/data"))
        .withValueBackReference("raw_contact_id", 0) // 关联第0个操作的ID
        .withValue("mimetype", "vnd.android.cursor.item/name")
        .withValue("data1", name)
        .build();
    ops.add(nameOp);
    // 3. 插入电话
    ContentProviderOperation phoneOp = ContentProviderOperation.newInsert(
        Uri.parse("content://com.android.contacts/data"))
        .withValueBackReference("raw_contact_id", 0)
        .withValue("mimetype", "vnd.android.cursor.item/phone_v2")
        .withValue("data1", phone)
        .withValue("data2", "2")
        .build();
    ops.add(phoneOp);
}

```

4. 查询联系人（获取姓名和电话）

```

private void queryContacts() {
    ContentResolver resolver = getContentResolver();
    // 查询data表，条件：mimetype是姓名或电话
}

```

```

Uri uri = Uri.parse("content://com.android.contacts/data");
Cursor cursor = resolver.query(
    uri,
    new String[]{"raw_contact_id", "data1", "mimetype"}, // 要查询的列
    null, null, null
);
if (cursor != null) {
    Map<Long, Contact> contactMap = new HashMap<>(); // 用Map暂存联系人 (ID→联系人对象)
    while (cursor.moveToNext()) {
        long contactId = cursor.getLong(cursor.getColumnIndex("raw_contact_id"));
        String data = cursor.getString(cursor.getColumnIndex("data1"));
        String mimetype = cursor.getString(cursor.getColumnIndex("mimetype"));
        // 从Map中获取或创建联系人对象
        Contact contact = contactMap.get(contactId);
        if (contact == null) {
            contact = new Contact();
            contactMap.put(contactId, contact);
        }
        // 判断数据类型，分别设置姓名和电话
        if ("vnd.android.cursor.item/name".equals(mimetype)) {
            contact.name = data;
        } else if ("vnd.android.cursor.item/phone_v2".equals(mimetype)) {
            contact.phone = data;
        }
    }
    cursor.close();
    // 显示查询结果
    StringBuilder sb = new StringBuilder();
    for (Contact contact : contactMap.values()) {
        sb.append("姓名：").append(contact.name).append(", 电话：").append(contact.phone).append("\n");
    }
    TextView tvResult = findViewById(R.id.tv_contacts);
    tvResult.setText(sb.toString());
}
// 联系人实体类（用于暂存数据）
class Contact {
    String name;

```

```
String phone;  
}
```

二、监听短信内容：实时获取验证码

很多 APP 需要自动获取短信验证码，这就需要监听系统短信数据库的变化。

1. 权限准备

```
<uses-permission android:name="android.permission.READ_SMS" /> <!-- 读短信权限 -->
```

同样需要动态申请（Android 6.0+）。

2. 用 ContentObserver 监听短信

```
private SmsObserver smsObserver;  
@Override  
protected void onCreate(Bundle savedInstanceState) {  
    super.onCreate(savedInstanceState);  
    setContentView(R.layout.activity_sms);  
    // 1. 创建短信观察者  
    smsObserver = new SmsObserver(new Handler());  
    // 2. 注册观察者，监听短信数据库变化  
    getContentResolver().registerContentObserver(  
        Uri.parse("content://sms/inbox"), // 监听收件箱  
        true, // 监听子目录  
        smsObserver  
    );  
}  
// 自定义短信观察者  
class SmsObserver extends ContentObserver {  
    public SmsObserver(Handler handler) {  
        super(handler);  
    }  
    // 短信数据库变化时调用  
    @Override
```

```

public void onChange(boolean selfChange, Uri uri) {
    super.onChange(selfChange, uri);
    if (uri.toString().contains("content://sms/inbox")) {
        // 读取最新收到的短信
        readLatestSms();
    }
}
// 读取最新的短信
private void readLatestSms() {
    ContentResolver resolver = getContentResolver();
    Cursor cursor = resolver.query(
        Uri.parse("content://sms/inbox"),
        new String[]{"address", "body", "date"}, // 发件人、内容、时间
        null, null, "date DESC" // 按时间倒序（最新的在前）
    );
    if (cursor != null && cursor.moveToFirst()) {
        String address = cursor.getString(cursor.getColumnIndex("address")); // 发件人号码
        String body = cursor.getString(cursor.getColumnIndex("body")); // 短信内容
        cursor.close();
        // 显示短信（实际项目中可提取验证码，如从body中匹配6位数字）
        runOnUiThread(() -> {
            TextView tvSms = findViewById(R.id.tv_sms);
            tvSms.setText("发件人： " + address + "\n内容： " + body);
        });
    }
}
// 页面销毁时注销观察者
@Override
protected void onDestroy() {
    super.onDestroy();
    if (smsObserver != null) {
        getContentResolver().unregisterContentObserver(smsObserver);
    }
}
}

```

三、跳转相册选择图片

APP 中更换头像、上传图片时，通常会让用户从相册选择，步骤如下：

1. 权限准备（Android 10 以下）

```
<uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE" /> <!-- 读
存储 -->
```

Android 10 及以上访问相册无需权限（通过系统 API 跳转）。

2. 跳转相册并获取图片

```
private static final int REQUEST_PICK_IMAGE = 101; // 请求码
private ImageView ivAvatar; // 用于显示选中的图片
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_album);
    ivAvatar = findViewById(R.id.iv_avatar);
    // 点击按钮跳转相册
    findViewById(R.id.btn_pick_image).setOnClickListener(v -> {
        // 创建意图：打开相册
        Intent intent = new Intent(Intent.ACTION_PICK);
        intent.setType("image/*"); // 类型：所有图片
        startActivityForResult(intent, REQUEST_PICK_IMAGE); // 启动并等待结果
    });
}
// 接收相册返回的结果
@Override
protected void onActivityResult(int requestCode, int resultCode, @Nullable Intent data) {
    super.onActivityResult(requestCode, resultCode, data);
    if (requestCode == REQUEST_PICK_IMAGE && resultCode == RESULT_OK && data != null) {
        // 获取选中图片的URI
        Uri imageUri = data.getData();
        if (imageUri != null) {
            // 显示图片到ImageView
            ivAvatar.setImageURI(imageUri);
        }
    }
}
```

四、发送短信：调用系统短信功能或直接发送

1. 调用系统短信界面（用户手动发送）

这种方式不需要权限，跳转到系统短信 APP，自动填充收件人和内容：

```
private void sendSmsBySystem() {  
    String phone = "13800138000"; // 收件人  
    String content = "你好，这是一条测试短信！ "; // 短信内容  
    Intent intent = new Intent(Intent.ACTION_SENDTO);  
    intent.setData(Uri.parse("smsto:" + phone)); // 指定收件人  
    intent.putExtra("sms_body", content); // 填充内容  
    startActivity(intent);  
}
```

2. 直接发送短信（无需用户操作）

需要发送权限，且用户可能看不到发送记录，谨慎使用：

步骤 1：添加权限

```
<uses-permission android:name="android.permission.SEND_SMS" /> <!-- 发送短信权限 -->
```

需要动态申请。

步骤 2：代码实现

```
private void sendSmsDirectly() {  
    String phone = "13800138000";  
    String content = "这是直接发送的短信！ ";  
    // 获取短信管理器  
    SmsManager smsManager = SmsManager.getDefault();  
    // 短信内容过长时拆分（超过70字）
```



```
List<String> messages = smsManager.divideMessage(content);
for (String msg : messages) {
    // 发送短信（最后一个参数是发送状态回调，可传null）
    smsManager.sendTextMessage(phone, null, msg, null, null);
}
Toast.makeText(this, "短信发送成功", Toast.LENGTH_SHORT).show();
}
```

总结

这节课咱们学了 6 个与系统交互的核心功能，都是实际开发高频使用的：

1. 操作联系人：

- 单个添加：先插raw_contacts表，再插data表存姓名和电话；
- 批量添加：用ContentProviderOperation高效处理；
- 查询：从data表筛选姓名和电话类型的数据。

2. 监听短信：

- 用ContentObserver监听sms/inbox数据库变化，实时获取新短信；
- 可从短信内容中提取验证码（正则匹配）。

3. 跳转相册：

- 用Intent.ACTION_PICK跳转，通过onActivityResult获取选中图片的 URI。

4. 发送短信：

- 调用系统界面：安全，无需权限，用户可见；
- 直接发送：需要SEND_SMS权限，适合自动发送验证码。

这些功能都需要注意权限申请（尤其是危险权限的动态申请）。课后可以做一个 "通讯录管理 APP"， - 实现添加、查询联系人，再加上发送短信给联系人的功能，巩固今天的知识！有问题随时问～

（注：文档部分内容可能由 AI 生成）